

Exp No:1	Candidate Elimination Algorithm
Date:	

AIM

To implement the Candidate Elimination Algorithm for learning a target concept and to determine the Specific Boundary (S) and General Boundary (G) of the version space using given training examples.

ALGORITHM

1. Initialize the most specific hypothesis S with $\emptyset$ values.
2. Initialize the most general hypothesis G with ? values.
3. For each training example:
 - If positive:
 - Generalize S minimally to include the example.
 - Remove inconsistent hypotheses from G.
 - If negative:
 - Specialize G minimally to exclude the example.
 - Remove inconsistent hypotheses.
4. Prune G to keep only maximally general hypotheses.
5. Repeat until all examples are processed.

PROGRAM

```
# Candidate Elimination Algorithm for Remote Work Approval
```

```
# Attributes:
```

```
# [Role, Experience, Performance, Internet, Location]
```

```
# Training data
```

```
training_data = [
    ("Technical", "Senior", "Excellent", "Good", "Urban", "Yes"),
    ("Technical", "Junior", "Excellent", "Good", "Urban", "Yes"),
    ("Non-Technical", "Junior", "Average", "Poor", "Rural", "No"),
    ("Technical", "Senior", "Average", "Good", "Rural", "No"),
    ("Technical", "Senior", "Excellent", "Good", "Rural", "Yes")
]
```

```
# Initialize S and G
```

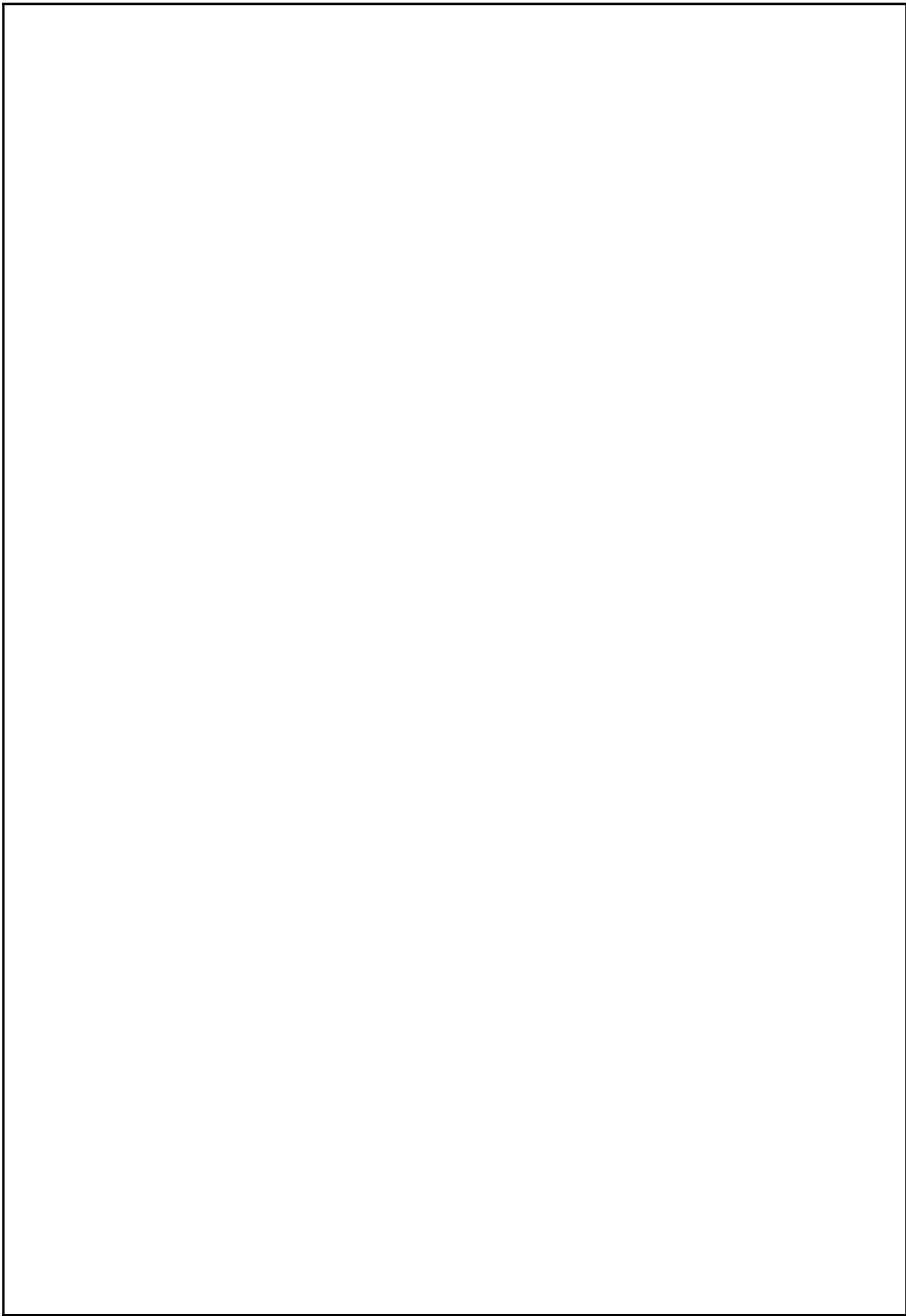
```
num_attributes = len(training_data[0][0])
```

```
S = ["∅"] * num_attributes
```

```
G = [{"?"] * num_attributes]
```

```
def is_consistent(hypothesis, instance):
```

```
    for h, i in zip(hypothesis, instance):
```



```

        if h != "?" and h != i:
            return False
        return True

def more_general(h1, h2):
    return all(x == "?" or x == y for x, y in zip(h1, h2))

print("\nInitial Specific Hypothesis (S):", S)
print("Initial General Hypothesis (G):", G)

# Process each training example
for idx, (instance, label) in enumerate(training_data):
    print(f"\nProcessing Example {idx+1}: {instance} -> {label}")

    if label == "Yes": # Positive example
        # Remove inconsistent hypotheses from G
        G = [g for g in G if is_consistent(g, instance)]

        # Generalize S
        for i in range(num_attributes):
            if S[i] == "∅":
                S[i] = instance[i]
            elif S[i] != instance[i]:
                S[i] = "?"

    else: # Negative example
        new_G = []
        for g in G:
            if is_consistent(g, instance):
                for i in range(num_attributes):
                    if g[i] == "?":
                        if S[i] != "?" and S[i] != "∅":
                            new_h = g.copy()
                            new_h[i] = S[i]
                            if is_consistent(new_h, instance) is False:
                                new_G.append(new_h)
            else:
                new_G.append(g)

        G = new_G

# Remove overly specific hypotheses in G
G = [g for g in G if more_general(g, S)]

print("Specific Boundary (S):", S)
print("General Boundary (G):", G)

# Final Version Space
print("\nFinal Specific Boundary (S):", S)
print("Final General Boundary (G):", G)

```

OUTPUT:

```

PS C:\Users\kavim> py "C:\Users\kavim\OneDrive\Desktop\Study Material\ML\Lab\Exp01\CandidateElimination.py"

Initial Specific Hypothesis (S): ['∅', '∅', '∅', '∅', '∅']
Initial General Hypothesis (G): [['?', '?', '?', '?', '?']]

Processing Example 1: ['Technical', 'Senior', 'Excellent', 'Good', 'Urban'] -> Yes
Specific Boundary (S): ['Technical', 'Senior', 'Excellent', 'Good', 'Urban']
General Boundary (G): [['?', '?', '?', '?', '?']]

Processing Example 2: ['Technical', 'Junior', 'Excellent', 'Good', 'Urban'] -> Yes
Specific Boundary (S): ['Technical', '?', 'Excellent', 'Good', 'Urban']
General Boundary (G): [['?', '?', '?', '?', '?']]

Processing Example 3: ['Non-Technical', 'Junior', 'Average', 'Poor', 'Rural'] -> No
Specific Boundary (S): ['Technical', '?', 'Excellent', 'Good', 'Urban']
General Boundary (G): [['Technical', '?', '?', '?', '?'], ['?', '?', 'Excellent', '?', '?'], ['?', '?', '?', 'Urban']]

Processing Example 4: ['Technical', 'Senior', 'Average', 'Good', 'Rural'] -> No
Specific Boundary (S): ['Technical', '?', 'Excellent', 'Good', 'Urban']
General Boundary (G): [['Technical', '?', '?', '?', '?'], ['Technical', '?', '?', '?', 'Urban'], ['?', '?', '?', 'Good', 'Urban'], ['?', '?', '?', '?', 'Urban']]

Processing Example 5: ['Technical', 'Senior', 'Excellent', 'Good', 'Rural'] -> Yes
Specific Boundary (S): ['Technical', '?', 'Excellent', 'Good', '?']
General Boundary (G): [['Technical', '?', 'Excellent', '?', '?'], ['?', '?', 'Excellent', '?', '?'], ['?', '?', '?', 'Urban']]

Final Specific Boundary (S): ['Technical', '?', 'Excellent', 'Good', '?']
Final General Boundary (G): [['Technical', '?', 'Excellent', '?', '?'], ['?', '?', 'Excellent', '?', '?'], ['?', '?', '?', 'Urban']]

Prediction for ['Technical', 'Senior', 'Excellent', 'Good', 'Urban']
Remote Work Approved? Yes
PS C:\Users\kavim>

```

```

# Prediction
test_instance = ["Technical", "Senior", "Excellent", "Good", "Urban"]

prediction = is_consistent(S, test_instance) and any(
    is_consistent(g, test_instance) for g in G
)

print("\nPrediction for", test_instance)
print("Remote Work Approved?" , "Yes" if prediction else "No")

```

Preparation	20	
Implementation	30	
Viva	15	
Record	10	
Total	75	

RESULT:

Thus, the Candidate Elimination Algorithm was successfully implemented to determine the Specific Boundary (S) and General Boundary (G) of the version space and uploaded to GitHub.

GitHub Link : <https://github.com/KavimugilRajasekar/MachineLearningLab.git>